ChatGPT

사이킷런 자체데이터 타이타닉으로 분류모델 제작 텐서플로우 제작, 함수없이, 모델튜닝, 성능평가

사이킷런의 타이타닉 데이터셋을 사용하여 분류 모델을 구축하고, TensorFlow를 사용해 모델을 만들고 튜닝하는 방법을 설명드리겠습니다. 이 예제에서는 함수 없이 모든 과정을 단계별로 진행합니다.

1. **데이터셋 로드 및 전처리**
   - `scikit-learn`의 `load_titanic` 함수는 제공되지 않으므로, `pandas`와 `seaborn` 라이브러리로 타이타닉 데이터를 로드하고 전처리합니다.
2. **모델 구축**
   - TensorFlow와 Keras를 이용하여 신경망 모델을 구축합니다.
3. **모델 튜닝**
   - 하이퍼파라미터 튜닝을 진행하여 모델 성능을 개선합니다.
4. **모델 평가**
   - 모델의 성능을 평가합니다.

1. 데이터셋 로드 및 전처리

```python
import pandas as pd
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

# 타이타닉 데이터셋 로드 (seaborn을 통해 제공)
titanic = sns.load_dataset('titanic')

# NaN 값 처리: 'age'와 'embarked' 열에 NaN값을 중앙값으로 채움
titanic['age'].fillna(titanic['age'].median(), inplace=True)
titanic['embarked'].fillna(titanic['embarked'].mode()[0], inplace=True)

# 'sex', 'embarked', 'class', 'embark_town' 등을 더미 변수로 변환
titanic = pd.get_dummies(titanic, columns=['sex', 'embarked', 'class', 'embark_town'], drop_
first=True)

# 나이, 요금, 승객수 등을 사용할 특성 선택
features = ['age', 'fare', 'sibsp', 'parch', 'sex_male', 'embarked_C', 'embarked_Q', 'class_
Third', 'class_First']
X = titanic[features]

# 종속 변수: 생존 여부
y = titanic['survived']

# 데이터 분할: 훈련 데이터 80%, 테스트 데이터 20%
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# 특성 정규화 (표준화)
```

```
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
```

2. 모델 구축 (TensorFlow/Keras 사용)

```python
import tensorflow as tf
from tensorflow.keras import layers, models

# 모델 구축
model = models.Sequential()

# 입력층: 8개의 특성 (입력 뉴런)
model.add(layers.InputLayer(input_shape=(X_train.shape[1],)))

# 히든층: 64개의 뉴런, ReLU 활성화 함수
model.add(layers.Dense(64, activation='relu'))

# 출력층: 1개의 뉴런, sigmoid 활성화 함수 (이진 분류)
model.add(layers.Dense(1, activation='sigmoid'))

# 모델 컴파일
model.compile(optimizer='adam', loss='binary_crossentropy', metrics=['accuracy'])
```

3. 모델 훈련 및 튜닝

```python
# 모델 훈련
history = model.fit(X_train, y_train, epochs=30, batch_size=32, validation_split=0.2, verbos
e=1)

# 훈련 과정에서의 정확도와 손실을 시각화하여 성능 확인
import matplotlib.pyplot as plt

# 훈련 과정 시각화 (정확도)
plt.plot(history.history['accuracy'], label='train accuracy')
plt.plot(history.history['val_accuracy'], label='validation accuracy')
plt.title('Model Accuracy')
plt.xlabel('Epoch')
plt.ylabel('Accuracy')
plt.legend(loc='best')
plt.show()

# 훈련 과정 시각화 (손실)
plt.plot(history.history['loss'], label='train loss')
plt.plot(history.history['val_loss'], label='validation loss')
plt.title('Model Loss')
plt.xlabel('Epoch')
plt.ylabel('Loss')
plt.legend(loc='best')
plt.show()
```

4. 모델 평가

```python
# 모델 평가: 테스트 데이터셋을 사용
test_loss, test_accuracy = model.evaluate(X_test, y_test)
print(f"Test Accuracy: {test_accuracy:.4f}")
```

5. 하이퍼파라미터 튜닝

모델 성능을 향상시키기 위해 하이퍼파라미터를 튜닝할 수 있습니다. 예를 들어, `epochs`, `batch_size`, `hidden layers` 등의 파라미터를 변경하면서 성능을 개선할 수 있습니다. 하이퍼파라미터 튜닝은 `GridSearchCV` 또는 `RandomizedSearchCV`와 같은 방법을 사용할 수 있습니다. 하지만 Keras와 TensorFlow에서 직접 튜닝하려면 `Keras Tuner`와 같은 라이브러리를 사용할 수 있습니다.

간단한 하이퍼파라미터 튜닝을 진행하려면, 여러 가지 조합을 시도하고, 각 실험의 성능을 확인해야 합니다. 이를 통해 최적의 파라미터를 선택할 수 있습니다.

6. 예측 및 결과 해석

```python
# 예측
y_pred = model.predict(X_test)

# 예측 값 이진화 (0.5를 기준으로)
y_pred_binary = (y_pred > 0.5).astype(int)

# 정확도 계산
from sklearn.metrics import accuracy_score, confusion_matrix

accuracy = accuracy_score(y_test, y_pred_binary)
print(f"Accuracy: {accuracy:.4f}")

# 혼동 행렬
cm = confusion_matrix(y_test, y_pred_binary)
print("Confusion Matrix:")
print(cm)
```

결론

위의 코드에서는 TensorFlow를 사용하여 타이타닉 데이터셋에 대해 분류 모델을 구축하고, 모델 성능을 평가하는 과정을 다루었습니다. 하이퍼파라미터 튜닝을 위해 `Keras Tuner`를 활용하면 좀 더 체계적으로 모델을 최적화할 수 있습니다.

분류리포트, 재현율 정밀도중 중점을 둔 이유와 함께 설명

분류 모델을 평가할 때 중요한 지표로 **정확도 (Accuracy)**, **정밀도 (Precision)**, **재현율 (Recall)**, **F1-Score** 등을 사용할 수 있습니다. 각 지표는 모델이 예측을 얼마나 잘 했는지에 대한 서로 다른 측면을 제공합니다. **정밀도**와 **재현율**은 특

히 불균형한 데이터셋에서 중요하게 여겨지며, 둘 사이에서 균형을 맞추는 것이 핵심입니다.

1. 분류 리포트 (Classification Report)

`scikit-learn`에서는 `classification_report`를 사용하여 여러 가지 중요한 성능 지표를 제공할 수 있습니다. 이 보고서는 각 클래스에 대해 다음의 지표를 제공합니다:

- **정밀도 (Precision)**: 모델이 양성이라고 예측한 샘플 중 실제로 양성인 샘플의 비율.

$$\text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FP}}$$

  여기서 TP는 **True Positive**(정답으로 양성으로 예측한 경우), FP는 **False Positive**(잘못 양성으로 예측한 경우)입니다.

- **재현율 (Recall)**: 실제 양성 샘플 중 모델이 양성으로 정확히 예측한 비율.

$$\text{Recall} = \frac{\text{TP}}{\text{TP} + \text{FN}}$$

  여기서 FN은 **False Negative**(실제 양성인데 잘못 음성으로 예측한 경우)입니다.

- **F1-Score**: 정밀도와 재현율의 조화 평균으로, 두 지표의 균형을 맞추는 지표입니다.

$$\text{F1-Score} = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$$

2. 정밀도와 재현율 중 하나를 중점 두는 이유

2.1 정밀도 (Precision)에 중점을 두는 이유

정밀도가 중요한 경우는 **모델이 잘못된 양성 예측을 최소화해야 할 때**입니다. 예를 들어, 이메일 스팸 필터링에서 "스팸"으로 분류된 이메일이 실제로 스팸이 아니면 사용자가 중요한 이메일을 놓칠 수 있습니다. 이런 경우, **잘못된 양성 예측 (False Positive)** 을 최소화하는 것이 중요하므로 정밀도에 중점을 둡니다.

- **예시**: 의료 진단에서 "암"이라고 예측하는 경우, 잘못된 양성 예측이 생기면 불필요한 추가 검사나 치료가 이루어질 수 있습니다. 이 경우 정밀도가 높으면 실제로 암이 아닌 사람을 암으로 잘못 진단하는 빈도를 낮출 수 있습니다.

2.2 재현율 (Recall)에 중점을 두는 이유

재현율이 중요한 경우는 **모델이 실제 양성을 놓치지 않도록 해야 할 때**입니다. 즉, 실제 양성 클래스에 대한 예측을 놓치지 않기 위해 중점을 둡니다. 예를 들어, 질병을 탐지하는 시스템에서는 환자가 실제로 병에 걸렸다면 이를 놓쳐서는 안 되므로 재현율에 중점을 둡니다.

- **예시**: 암 검사에서 재현율이 중요합니다. 실제 암 환자를 놓치면 생명이 위태로워질 수 있습니다. 이 경우 재현율을 높여서 실제로 암이 있는 사람을 놓치지 않도록 하는 것이 중요합니다.

2.3 정밀도와 재현율의 트레이드오프

정밀도와 재현율은 보통 상반된 관계에 있습니다. 즉, 정밀도를 높이면 재현율이 낮아지고, 반대로 재현율을 높이면 정밀도가 낮아질 수 있습니다. 이를 **모델의 트레이드오프**라고 합니다. 예를 들어, 어떤 상황에서는 양성 예측을 많이 하여 재현율을 높이려고 할 때, 잘못된 예측이 많아져 정밀도가 떨어질 수 있습니다.

따라서 **F1-Score**는 정밀도와 재현율을 동시에 고려하여 이 둘 사이의 균형을 맞추는 지표로 사용됩니다. F1-Score는 둘 중 어느 한 쪽이 너무 낮은 경우 모델 성능이 나쁘다고 판단할 수 있습니다.

3. 어떤 지표에 중점을 둘지 결정하는 방법

- **정밀도에 중점을 둬야 하는 경우**:
  - 잘못된 양성 예측으로 인한 비용이 큰 경우.
  - 예: 스팸 필터링, 금융 사기 탐지, 불법 콘텐츠 탐지 등.
- **재현율에 중점을 둬야 하는 경우**:
  - 실제 양성을 놓치는 것이 큰 문제가 되는 경우.
  - 예: 질병 진단, 사고 예측, 범죄 탐지 등.

4. 실제 타이타닉 데이터에서 적용

타이타닉 데이터에서는 **생존자 예측**이 중요한 문제입니다. 이 문제에서 **재현율**에 중점을 두는 것이 좋을 수 있습니다. 왜냐하면, 모델이 생존자를 놓치는 경우 (False Negative)가 큰 피해로 이어질 수 있기 때문입니다. 예를 들어, **생존자**를 놓치면 실제로 구조될 수 있었던 사람을 구하지 못하는 상황이 발생할 수 있습니다.

따라서 타이타닉 생존자 예측 문제에서는 재현율을 중점적으로 고려하고, F1-Score를 통해 정밀도와 재현율의 균형을 맞추는 것이 바람직합니다.

5. 분류 리포트 출력 예시

```python
from sklearn.metrics import classification_report

# 분류 리포트 출력
report = classification_report(y_test, y_pred_binary)
print(report)
```

이렇게 출력된 분류 리포트에서 **정밀도, 재현율, F1-Score**를 확인하여 모델의 성능을 평가하고, 개선할 부분을 찾아낼 수 있습니다.

결론

- **정밀도**는 잘못된 양성 예측을 줄이는 데 중요하고, **재현율**은 실제 양성을 놓치지 않도록 하는 데 중요합니다.
- 타이타닉 예측과 같은 경우 **재현율**을 더 중시할 수 있으며, 이 두 지표 간의 균형을 맞추는 것이 핵심입니다.
- **F1-Score**를 통해 두 지표를 동시에 고려하는 것이 모델 성능을 최적화하는 데 유용합니다.

분류리포트 시각화 코드까지,

분류 리포트를 시각화하는 방법으로 **히트맵(Heatmap)** 또는 **막대 그래프(Bar Chart)** 등을 사용할 수 있습니다. `seaborn`과 `matplotlib`를 사용하면 직관적으로 분류 성능을 시각화할 수 있습니다.

여기서는 **혼동 행렬(Confusion Matrix)**과 **정밀도, 재현율, F1-Score**를 각각 시각화하는 방법을 설명합니다.

1. 혼동 행렬(Confusion Matrix) 시각화

혼동 행렬은 모델의 예측 성능을 이해하는 데 유용한 도구로, True Positive (TP), False Positive (FP), False Negative (FN), True Negative (TN)을 포함합니다. 이 행렬을 시각화하면 모델이 얼마나 잘 예측했는지, 어떤 종류의 오류를 발생시켰는지 쉽게 알 수 있습니다.

```python
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.metrics import confusion_matrix

# 혼동 행렬 계산
cm = confusion_matrix(y_test, y_pred_binary)

# 혼동 행렬을 히트맵으로 시각화
plt.figure(figsize=(7, 5))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', xticklabels=['Predicted No', 'Predicted Y
es'], yticklabels=['Actual No', 'Actual Yes'])
plt.title('Confusion Matrix')
plt.xlabel('Predicted')
plt.ylabel('Actual')
plt.show()
```

이 코드는 **혼동 행렬**을 히트맵으로 시각화합니다. `**annot=True**`로 숫자를 표시하고, `**fmt='d'**`는 정수 형식으로 표시하도록 합니다.

2. 정밀도, 재현율, F1-Score 시각화 (Bar Chart)

`**classification_report**`에서 제공하는 정밀도, 재현율, F1-Score를 막대 그래프 형식으로 시각화할 수 있습니다. 아래 코드는 각 클래스별로 정밀도, 재현율, F1-Score를 표시하는 방법을 보여줍니다.

```python
from sklearn.metrics import classification_report
import pandas as pd

# 분류 리포트
report = classification_report(y_test, y_pred_binary, output_dict=True)

# 정밀도, 재현율, F1-Score를 DataFrame으로 변환
report_df = pd.DataFrame(report).transpose()

# 시각화: 정밀도, 재현율, F1-Score에 대한 막대 그래프
report_df.drop('accuracy', axis=1, inplace=True)  # 'accuracy' 열은 제외
report_df.plot(kind='bar', figsize=(10, 6), color=['#1f77b4', '#ff7f0e', '#2ca02c'])

# 그래프 제목 및 레이블 설정
plt.title('Precision, Recall, and F1-Score for Each Class')
plt.xlabel('Class')
plt.ylabel('Score')
plt.xticks(rotation=0)
plt.legend(title='Metrics', labels=['Precision', 'Recall', 'F1-Score'])
plt.show()
```

이 코드는 `**classification_report**`에서 나온 정밀도, 재현율, F1-Score를 **막대 그래프**로 시각화합니다. 각 클래스에 대해 세 가지 지표를 비교할 수 있습니다.

3. 전체 코드 예시

위의 코드를 하나로 합쳐서, 타이타닉 데이터셋을 기준으로 **혼동 행렬**과 **정밀도, 재현율, F1-Score 시각화**까지 전체적인 분석을 진행하는 예시는 다음과 같습니다:

```python
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import classification_report, confusion_matrix
from tensorflow.keras import layers, models

# 데이터 로드 및 전처리 (타이타닉 데이터셋)
titanic = sns.load_dataset('titanic')
titanic['age'].fillna(titanic['age'].median(), inplace=True)
titanic['embarked'].fillna(titanic['embarked'].mode()[0], inplace=True)
titanic = pd.get_dummies(titanic, columns=['sex', 'embarked', 'class', 'embark_town'], drop_
first=True)
features = ['age', 'fare', 'sibsp', 'parch', 'sex_male', 'embarked_C', 'embarked_Q', 'class_
Third', 'class_First']
X = titanic[features]
y = titanic['survived']
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# 모델 구축
model = models.Sequential()
model.add(layers.InputLayer(input_shape=(X_train.shape[1],)))
model.add(layers.Dense(64, activation='relu'))
model.add(layers.Dense(1, activation='sigmoid'))
model.compile(optimizer='adam', loss='binary_crossentropy', metrics=['accuracy'])

# 모델 훈련
history = model.fit(X_train, y_train, epochs=30, batch_size=32, validation_split=0.2, verbos
e=0)

# 예측
y_pred = model.predict(X_test)
y_pred_binary = (y_pred > 0.5).astype(int)

# 1. 혼동 행렬 시각화
cm = confusion_matrix(y_test, y_pred_binary)
plt.figure(figsize=(7, 5))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', xticklabels=['Predicted No', 'Predicted Y
es'], yticklabels=['Actual No', 'Actual Yes'])
plt.title('Confusion Matrix')
plt.xlabel('Predicted')
plt.ylabel('Actual')
plt.show()

# 2. 정밀도, 재현율, F1-Score 시각화
report = classification_report(y_test, y_pred_binary, output_dict=True)
report_df = pd.DataFrame(report).transpose()
report_df.drop('accuracy', axis=1, inplace=True)
report_df.plot(kind='bar', figsize=(10, 6), color=['#1f77b4', '#ff7f0e', '#2ca02c'])

plt.title('Precision, Recall, and F1-Score for Each Class')
plt.xlabel('Class')
plt.ylabel('Score')
plt.xticks(rotation=0)
plt.legend(title='Metrics', labels=['Precision', 'Recall', 'F1-Score'])
plt.show()
```

설명:

- **혼동 행렬 시각화**: 예측과 실제 값을 비교하여 모델의 예측 오류를 시각적으로 확인할 수 있습니다.
- **정밀도, 재현율, F1-Score 시각화**: 각 클래스에 대한 성능 지표를 시각적으로 비교할 수 있습니다. `precision`, `recall`, `f1-score` 값이 각 클래스에 대해 어떻게 다른지 확인할 수 있습니다.

위의 코드를 실행하면, **혼동 행렬**과 **정밀도, 재현율, F1-Score**에 대한 시각화를 통해 모델 성능을 한눈에 파악할 수 있습니다.

모델 저장 및 피클 필요한 부분 작업해서, 다시 모델 로드해서 새로운 데이터 추론

모델을 저장하고 로드하여 새로운 데이터에 대해 예측을 수행하는 방법을 설명하겠습니다. **TensorFlow/Keras** 모델을 저장하고, **피클(pickle)**을 사용하여 모델을 로드한 후, 새로운 데이터를 입력으로 사용하여 예측하는 과정을 포함합니다.

1. 모델 저장

Keras 모델을 저장하려면 `model.save()` 메서드를 사용하여 모델을 `.h5` 형식으로 저장할 수 있습니다. 피클은 Keras 모델을 저장하기에는 적합하지 않지만, `model.save()`를 사용하면 모델의 구조와 가중치를 포함한 모든 정보를 파일로 저장할 수 있습니다.

2. 모델 로드 및 새로운 데이터 추론

모델을 로드한 후, 새로운 데이터를 입력으로 예측을 수행합니다. 새로운 데이터는 기존 훈련 데이터와 동일한 전처리가 필요합니다.

전체 흐름:

1. **모델 저장**: `model.save()`를 사용하여 모델을 저장.
2. **모델 로드**: `tensorflow.keras.models.load_model()`을 사용하여 저장된 모델을 로드.
3. **새로운 데이터 예측**: 새 데이터에 대해 예측을 수행.

전체 코드 예시

```python
import pandas as pd
import seaborn as sns
import tensorflow as tf
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from tensorflow.keras import layers, models
import pickle
import numpy as np

# 타이타닉 데이터셋 로드 및 전처리
titanic = sns.load_dataset('titanic')
titanic['age'].fillna(titanic['age'].median(), inplace=True)
```

```
titanic['embarked'].fillna(titanic['embarked'].mode()[0], inplace=True)
titanic = pd.get_dummies(titanic, columns=['sex', 'embarked', 'class', 'embark_town'], drop_
first=True)
features = ['age', 'fare', 'sibsp', 'parch', 'sex_male', 'embarked_C', 'embarked_Q', 'class_
Third', 'class_First']
X = titanic[features]
y = titanic['survived']

# 데이터 분할
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# 특성 표준화
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# 모델 구축
model = models.Sequential()
model.add(layers.InputLayer(input_shape=(X_train.shape[1],)))
model.add(layers.Dense(64, activation='relu'))
model.add(layers.Dense(1, activation='sigmoid'))
model.compile(optimizer='adam', loss='binary_crossentropy', metrics=['accuracy'])

# 모델 훈련
model.fit(X_train, y_train, epochs=30, batch_size=32, validation_split=0.2, verbose=0)

# 모델 저장
model.save('titanic_model.h5')  # 모델을 'titanic_model.h5' 파일로 저장

# 피클로 스케일러 저장
with open('scaler.pkl', 'wb') as f:
    pickle.dump(scaler, f)

# 모델 및 스케일러 로드
loaded_model = tf.keras.models.load_model('titanic_model.h5')

# 스케일러 로드
with open('scaler.pkl', 'rb') as f:
    loaded_scaler = pickle.load(f)

# 새로운 데이터 (예시)
new_data = pd.DataFrame({
    'age': [29],
    'fare': [100],
    'sibsp': [0],
    'parch': [0],
    'sex_male': [1],
    'embarked_C': [0],
    'embarked_Q': [0],
    'class_Third': [0],
    'class_First': [1]
})

# 새로운 데이터 전처리
new_data_scaled = loaded_scaler.transform(new_data)

# 모델 예측
prediction = loaded_model.predict(new_data_scaled)

# 예측 결과 출력
print(f"Prediction (Survived=1, Not Survived=0): {prediction[0][0] > 0.5}")
```

코드 설명:

1. **모델 훈련 및 저장**:
    - 모델을 훈련시킨 후 `**model.save('titanic_model.h5')**`를 사용하여 모델을 `**.h5**` 파일로 저장합니다.

- 스케일러 객체는 피클로 저장합니다. `pickle.dump(scaler, f)`를 사용하여 훈련에 사용한 스케일러를 저장합니다.

2. **모델 및 스케일러 로드**:
   - `loaded_model = tf.keras.models.load_model('titanic_model.h5')`를 사용하여 저장된 모델을 로드합니다.
   - `pickle.load(f)`를 사용하여 저장된 스케일러를 로드합니다.
3. **새로운 데이터 추론**:
   - 새로운 데이터를 `new_data`로 정의하고, 이를 모델이 훈련된 방식대로 전처리합니다.
   - `new_data_scaled = loaded_scaler.transform(new_data)`로 새 데이터를 표준화합니다.
   - 모델 예측은 `loaded_model.predict(new_data_scaled)`로 진행하고, 예측된 결과가 0.5 이상이면 생존, 그렇지 않으면 사망으로 해석합니다.

3. 피클로 스케일러 저장 및 로드

피클을 사용하여 모델을 제외한 다른 객체 (예: 데이터 스케일러)를 저장하고 로드할 수 있습니다. 이는 모델이 훈련된 전처리 파이프라인을 동일하게 적용하기 위해 중요합니다.

```python
# 피클로 스케일러 저장
with open('scaler.pkl', 'wb') as f:
    pickle.dump(scaler, f)

# 피클로 스케일러 로드
with open('scaler.pkl', 'rb') as f:
    loaded_scaler = pickle.load(f)
```

결론

이 과정은 **모델 저장, 로드, 새로운 데이터에 대한 예측**을 쉽게 처리할 수 있게 해줍니다. 피클을 사용하여 모델과 함께 필요한 전처리 객체도 저장하고 로드할 수 있어 일관된 예측을 유지할 수 있습니다.